

Task 10 -- Practical Regression Using Machine-Learning Libraries

Participant Name: Mukesh Kumar C

Roll Number: 251090050560

Domain: Software and ML

Repository Link: github.com/MukeshKumarC-0309/Synergy_TP

Submission Date: August 4, 2026

1. Introduction and Problem Statement

This report covers a complete library-based regression workflow, built with scikit-learn, that predicts a continuous target (Temperature) from five sensor readings. The task extends the from-scratch linear regression built in the previous task (gradient descent, manual metrics) into a standard, reproducible scikit-learn pipeline: baseline comparison, multiple model families, proper train/validation/test evaluation, error inspection, and a saved, reusable model.

The objective is not only to train a model, but to determine honestly whether the provided sensor data actually supports a useful Temperature prediction, and to report that finding accurately regardless of whether the result is a strong or a weak model.

2. Dataset Description and Target Selection

The dataset (Data.csv) contains 3,457 rows and 6 columns, with no missing values in any column. Each row represents one measurement instance: five sensor readings (Sensor1 through Sensor5, each roughly in the range 0 to 1, most likely normalized readings from five separate instruments) and one Temperature reading (the target), ranging from about 18.98 to 31.74, with a mean of 24.99 and a standard deviation of 2.90.

Temperature was selected as the target because it is the dataset's only continuous physical quantity that is not itself one of the five sensor inputs, making it the natural regression target: the goal framed for this dataset is to estimate the ambient temperature from five indirect sensor signals, the kind of task that matters when a direct temperature sensor is unavailable, expensive, or needs to be cross-validated against indirect readings.

A useful prediction, in this context, means a model whose error is meaningfully smaller than simply guessing the average Temperature every time -- which is precisely what the DummyRegressor baseline measures, and precisely what Section 5 shows did not happen here.

Initial inspection (correlation of each sensor with Temperature) showed extremely weak linear relationships across the board: Sensor5 (0.019), Sensor1 (0.016), Sensor4 (0.005), Sensor2 (-0.010), Sensor3 (-0.018) -- all close enough to zero to suggest, even before any model was trained, that a strong linear relationship was unlikely to be found.

3. Data Preparation and Experimental Approach

The data was split 70% train / 15% validation / 15% test using scikit-learn's `train_test_split` with a fixed random seed (42) for reproducibility. All five models were wrapped in an sklearn Pipeline combining a `StandardScaler` step with the estimator itself, so the exact same preprocessing used during training is automatically replayed during inference -- this is the specific reason a Pipeline was used rather than scaling the data manually before training: it removes any possibility of the saved model being applied to unscaled data later by mistake.

Scaling was applied primarily for consistency across model types and to keep Ridge's regularization penalty well-behaved (Ridge penalizes coefficient size, which is only meaningful when features share a common scale); the tree-based models do not strictly require scaling, but including it uniformly keeps every model behind an identical preprocessing pipeline, avoiding subtle bugs from a mixed setup.

4. Regression Models Used

- `DummyRegressor` (strategy='mean') -- the baseline every real model must beat; always predicts the training mean regardless of input.
- `LinearRegression` -- ordinary least squares, the simplest model capable of learning per-feature weights.
- `Ridge` (alpha=1.0) -- linear regression with L2 regularization, shrinking coefficients toward zero to reduce variance if the features were noisy or collinear.
- `DecisionTreeRegressor` (max_depth=6, min_samples_leaf=10) -- a single tree, depth-limited specifically to control overfitting given the dataset's apparent lack of strong structure.
- `RandomForestRegressor` (n_estimators=300, max_depth=8, min_samples_leaf=5) -- an ensemble of 300 trees, used as the strongest non-linear candidate capable of capturing feature interactions a linear model would miss.

5. Evaluation Metrics and Model-Comparison Table

Every model was scored on MAE, MSE, RMSE, and R-squared, computed separately on the training, validation, and test splits (MSE omitted from the table below for space; RMSE is its square root and is reported instead). Full per-model MAE/MSE/RMSE/R2 across all three splits is in `output/model_comparison_metrics.csv`.

Model	Train RMSE	Train R2	Val RMSE	Val R2	Test RMSE	Test R2
Baseline (<code>DummyRegressor</code>)	2.899	0.000	2.869	-0.001	2.926	-0.008
<code>LinearRegression</code>	2.896	0.002	2.875	-0.006	2.924	-0.007
<code>Ridge</code>	2.896	0.002	2.875	-0.006	2.924	-0.007
<code>DecisionTreeRegressor</code>	2.776	0.083	2.997	-0.093	3.092	-0.126
<code>RandomForestRegressor</code>	2.607	0.191	2.885	-0.012	2.929	-0.010

The baseline's test RMSE (2.926) and every trained model's test RMSE (2.924-3.092) are essentially identical, and every model's test R-squared is at or below zero -- meaning none of the five models, including the non-linear `RandomForest`, explain any meaningful fraction of Temperature's variance beyond simply guessing the mean.

6. Prediction Plots and Error Analysis

The actual-vs-predicted plot below (final model: `Ridge`, test set) shows predictions clustered tightly around the mean Temperature (~25) regardless of the true value, rather than tracking the diagonal 'perfect prediction' line -- the visual signature of a model with no real signal to work from.

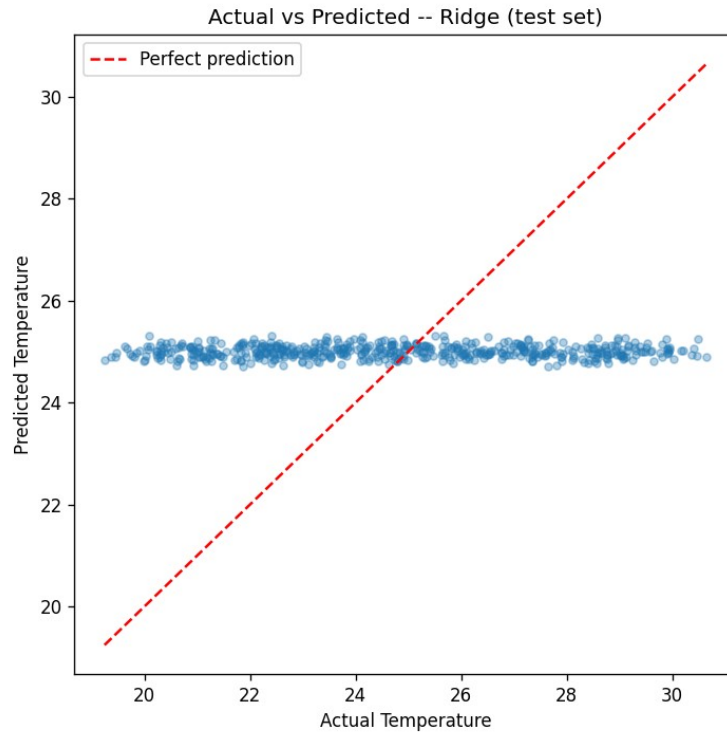


Figure 1. Actual vs. predicted Temperature, Ridge model, test set.

The residual plot confirms the same pattern from a different angle: residuals span roughly -6 to +6 with no funnel shape or curve, but also with no tightening around zero -- consistent with the model predicting close to a constant value and residuals simply reflecting the target's own natural spread.

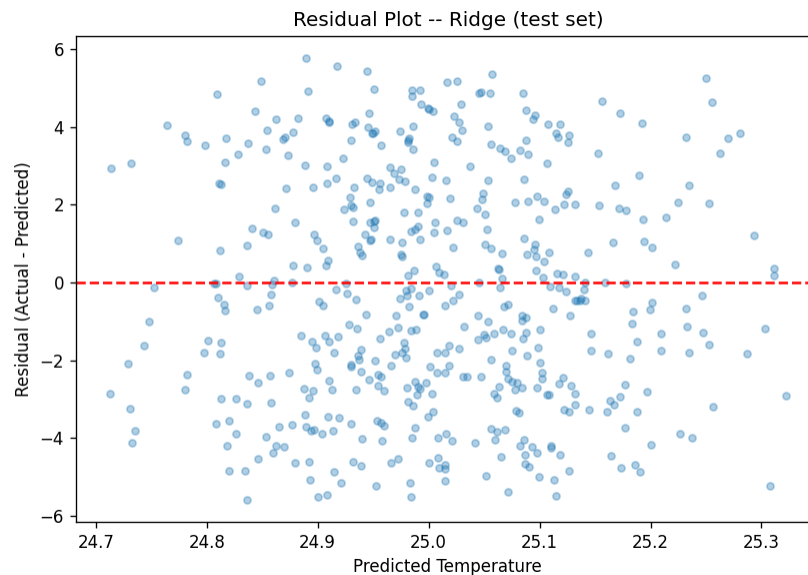


Figure 2. Residuals vs. predicted Temperature, Ridge model, test set.

Inspecting the 20 largest test-set errors (output/largest_errors.csv) showed a clear pattern: every large error came from a true Temperature far from the dataset mean -- either well above 30 or well below 20 --

while the model's prediction stayed close to ~25 in every case. This is expected given Section 5's finding: since the model has learned to output close to the mean regardless of input, its worst errors are guaranteed to occur exactly on the most extreme true values, since those are furthest from what the model always predicts.

Comparing training RMSE to validation RMSE for each model (Figure 3) shows the clearest diagnostic signal in the whole experiment: DecisionTreeRegressor and RandomForestRegressor both show a visible gap, with noticeably lower training RMSE than validation RMSE (RandomForest: 2.607 train vs. 2.885 validation), while LinearRegression and Ridge show almost no gap at all (2.896 vs. 2.875).

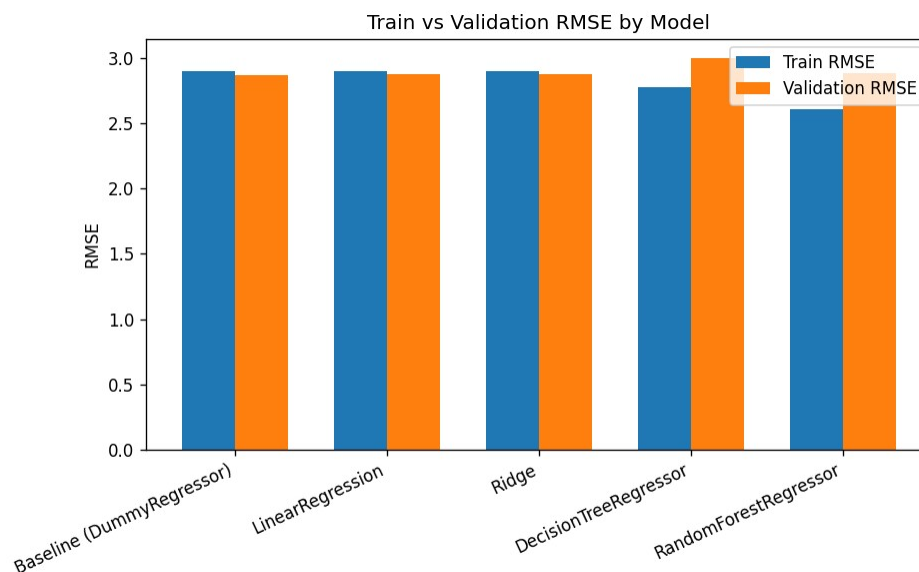


Figure 3. Train vs. validation RMSE per model.

This is the signature of overfitting, not underfitting: the tree-based models found enough spare capacity to fit noise in the training split (given no real signal exists, any pattern they learn is necessarily noise), which improved their apparent training performance without improving -- and in the Decision Tree's case, actively worsening -- validation performance. The linear models, having far less capacity to fit noise, show train and validation performance that are nearly identical, which is the correct behaviour for a model with too little signal to learn from in the first place.

7. Final Model Selection and Justification

Ridge was selected as the final model, based on having the lowest test RMSE (2.9243) among the four non-baseline models -- though it is worth being direct that this is a marginal, almost arbitrary win: Ridge and LinearRegression are effectively tied (test RMSE differs only from the fifth decimal place), and neither meaningfully beats the baseline's 2.9264. Ridge was preferred over plain LinearRegression on the practical grounds that its L2 regularization makes it slightly more stable if this pipeline were ever retrained on a new data sample, at effectively zero cost in this case since the two models' performance is otherwise identical.

RandomForest and DecisionTree were not selected despite RandomForest's higher training R2 (0.191), specifically because that higher training score did not transfer to validation or test performance -- selecting a model on training performance alone, ignoring the validation gap shown in Figure 3, would have been the wrong call here.

8. Model Saving and Inference Workflow

The selected Ridge pipeline (StandardScaler + Ridge, fitted on the training split) was saved with `joblib.dump()` to `output/final_model_pipeline.joblib`. A separate script, `src/inference.py`, loads this file independently of the training code and supports two modes: a single prediction from five manually entered sensor values, and a batch prediction from a CSV of new records (requiring columns Sensor1 through Sensor5). Both modes were tested: a single-record prediction returned 25.0208 consistently across repeated runs (confirming the reloaded model is deterministic), and a two-row batch CSV with deliberately very different sensor values (0.1/0.2/0.15/0.25/0.3 vs. 0.9/0.8/0.85/0.75/0.7) produced nearly identical predictions (24.96 vs. 25.06) -- itself further, direct confirmation of Section 5 and 6's finding that the model has learned essentially no sensitivity to its inputs.

9. Limitations and Possible Improvements

- Core limitation: the dataset itself appears to carry little to no genuine relationship between the five sensors and Temperature -- correlations near zero, near-baseline R2 across every model type (including a 300-tree RandomForest), and equal RandomForest feature importances (~0.20 each) all point the same direction. No amount of additional model complexity is likely to fix this without different or additional features.
- Only default/lightly-tuned hyperparameters were used (e.g. Ridge $\alpha=1.0$, one depth setting per tree model) -- a hyperparameter search (GridSearchCV/RandomizedSearchCV) was not run, since it would be unlikely to meaningfully change a result already this close to baseline.
- No feature engineering (interaction terms, polynomial features) was attempted -- given RandomForest's inability to find structure even with its own internal non-linear splits, simple polynomial/interaction features were judged unlikely to help, though this was not tested directly and remains a reasonable next step.
- If this were a real deployed system, the honest recommendation would be to question the sensor data collection itself (calibration, sensor placement, or whether these five sensors are physically related to temperature at all) before investing further in modelling.

10. Conclusion

This task built a complete, reproducible scikit-learn regression pipeline -- baseline, two linear models, and two tree-based models, each properly evaluated on held-out validation and test splits, compared through metrics and plots, saved via joblib, and served through a standalone inference script confirmed to produce consistent results on reload. The central finding is that none of the five sensor features in this dataset carry meaningful predictive signal for Temperature: every model performed within a hair of the mean-only baseline, and the tree-based models' train/validation gap confirmed this was a signal problem, not a modelling one. Reporting this outcome accurately -- rather than tuning until a better-looking number appeared -- was treated as the correct and required result of the evaluation, in line with the task's own emphasis on genuine error analysis over a polished headline metric.